

Trabalho - Assembly

Biblioteca Assembly

Sistema de gerenciamento de biblioteca escrito em Assembly MIPS, executado no simulador MARS.

Repositório: <https://github.com/Caioohv/biblioteca-assembly>

1. Introdução

O bibliotecário precisa de um sistema para gerir sua biblioteca, sendo necessário saber quais livros estão cadastrados, quais estão disponíveis e quais estão emprestados. Duas regras de negócio devem ser garantidas:

- é impossível emprestar um livro que já está emprestado;
- é impossível devolver um livro que não está emprestado.

O programa foi implementado em Assembly MIPS e funciona por meio de um **menu de texto** exibido no console. A cada iteração o usuário escolhe uma das cinco operações disponíveis (cadastrar, emprestar, devolver, listar ou sair) e o programa executa a ação correspondente, retornando ao menu ao final. A execução só termina quando o usuário escolhe a opção `0` (Sair).

Os dados são mantidos em memória por meio de dois vetores paralelos: um para os códigos dos livros e outro para os respectivos estados. Não há persistência em disco - os dados existem apenas durante a execução.

2. Implementação

2.1 Estruturas de dados (segmento `.data`)

O estado do sistema é armazenado em três variáveis globais, além das mensagens de interface:

```
qtd_livros: .word 0
codigos:    .space 200    # 50 * 4 bytes
status:     .space 50     # 50 * 1 byte
```

- `qtd_livros` - contador de livros cadastrados (inteiro, 4 bytes). Também é usado como próximo código a ser atribuído.
- `codigos` - vetor de inteiros (50 posições × 4 bytes) que guarda o código de cada livro.
- `status` - vetor de bytes (50 posições × 1 byte) que guarda o estado de cada livro, alinhado por índice ao vetor `codigos`.

Convenção de status adotada: 1 = disponível e 0 = emprestado. Essa decisão foi tomada para uniformizar a leitura do estado em todas as rotinas (cadastro, empréstimo, devolução e listagem), evitando inconsistências.

A capacidade máxima foi fixada em **50 livros**, o que define tanto o tamanho dos vetores quanto a verificação de "biblioteca cheia".

2.2 Menu principal (`menu` / `opcaoinvalida` / `exit`)

O menu imprime as opções (syscall 4), lê um inteiro do usuário (syscall 5) e desvia para a rotina correspondente com uma cadeia de comparações `beq`. Qualquer valor fora do intervalo 0–4 cai em `opcaoinvalida`, que exibe a mensagem de erro e retorna ao menu. A opção 0 chama a syscall 10, encerrando o programa.

```
menu:
    li $v0, 4
    la $a0, msginicial
    syscall

    li $v0, 5
    syscall
    move $t0, $v0

    beq $t0, 0, exit
    beq $t0, 1, cadastrar
    beq $t0, 2, emprestar
    beq $t0, 3, devolver
    beq $t0, 4, listar
    j opcaoinvalida
```

2.3 Cadastro de livros (`cadastrar`)

O cadastro primeiro verifica se a biblioteca está cheia (`bge $t1, 50`); em caso positivo desvia para `biblioteca_cheia`. Caso contrário, lê o título do livro (syscall 8, apenas como interface - o título não é armazenado, pois a especificação exige apenas o código numérico). Em seguida:

1. incrementa `qtd_livros` , gerando o novo código (`t1 + 1`);
2. calcula o endereço `codigos[indice]` deslocando o índice em 2 bits (`sll` , equivalente a multiplicar por 4) e somando à base do vetor;
3. grava o código na posição calculada;
4. calcula `status[indice]` (soma direta, pois cada elemento tem 1 byte) e grava `1` (disponível);
5. imprime o código gerado e a mensagem de confirmação.

```
cadastar:
    #verifica se a biblioteca esta cheia
    lw $t1, qtd_livros
    bge $t1, 50, biblioteca_cheia

    #print msg
    li $v0, 4
    la $a0, msgcadastar
    syscall

    #le nome do livro
    li $v0, 8
    la $a0, livro_buffer
    li $a1, 100
    syscall

    #counter + 1
    lw $t1, qtd_livros
    addi $t2, $t1, 1
    sw $t2, qtd_livros

    la $t3, codigos      # puxa o array
    sll $t4, $t1, 2      # mult por 4 (int = 4)
    add $t4, $t3, $t4    # t4 = &codigos[indice]
    sw $t2, 0($t4)       # codigos[indice] = codigo

    la $t5, status
    add $t5, $t5, $t1    # t5 = &status[indice]
    li $t6, 1           # 1 = disponivel, 0 = emprestado
    sb $t6, 0($t5)      # status[indice] = disponivel
```

O código de um livro é sempre igual à sua posição de cadastro (1, 2, 3, ...), o que permite mapear diretamente o código informado pelo usuário para o índice do vetor (`índice = código - 1`) nas operações de empréstimo e devolução.

2.4 Listagem de livros (`listar`)

Percorre os livros de 0 até `qtd_livros - 1` com um laço (`listar_loop`). Para cada livro imprime o código (lido de `codigos[indice]`) e, conforme o valor de `status[indice]` , imprime `| Disponível (status 1)` ou `| Emprestado` (qualquer outro valor). Ao final do laço retorna ao menu.

```
listar_loop:
    beq $t1, $t0, menu    # se indice == total, acabou -> volta pro menu
    ...
    beq $t6, 1, listar_disponivel
    j listar_emprestado
```

2.5 Empréstimo (`emprestar`)

Lê o código do livro e o valida: precisa estar entre 1 e `qtd_livros` (`blt / bgt`), caso contrário desvia para `codigoinvalido` . Converte o código em índice (`código - 1`), lê o status atual e só efetua o empréstimo se o livro estiver disponível (`bne $t4, 1, emprestar_erro`). Ao emprestar, grava 0 (emprestado). Se o livro já estiver emprestado, exibe a mensagem de erro, essa verificação garante a regra "não emprestar livro já emprestado".

```
emprestar:
    ...
    #valida o codigo (1 ate qtd_livros)
    lw $t1, qtd_livros
    blt $t0, 1, codigoinvalido
    bgt $t0, $t1, codigoinvalido

    #pega &status[codigo - 1]
    addi $t2, $t0, -1
    la $t3, status
    add $t3, $t3, $t2
    lb $t4, 0($t3)

    #so empresta se estiver disponivel
    bne $t4, 1, emprestar_erro
    li $t5, 0
    sb $t5, 0($t3)    # status[indice] = emprestado
```

2.6 Devolução (`devolver`)

Segue a mesma lógica de validação do empréstimo. A diferença é a verificação: só devolve se o livro estiver emprestado (`bne $t4, 0, devolver_erro`); nesse caso

grava **1** (disponível). Se o livro não estiver emprestado, exibe a mensagem de erro, garantindo a regra "não devolver livro que não está emprestado".

```
devolver:
    ...
    #pega &status[codigo - 1]
    addi $t2, $t0, -1
    la $t3, status
    add $t3, $t3, $t2
    lb $t4, 0($t3)

    #so devolve se estiver emprestado (status == 0)
    bne $t4, 0, devolver_erro
    li $t5, 1
    sb $t5, 0($t3)      # status[indice] = disponivel
```

2.7 Decisões de projeto e casos omissos

- **Título do livro:** o enunciado especifica apenas código e status. O título é solicitado ao usuário por questão de usabilidade, mas não é armazenado nem utilizado nas demais operações.
- **Geração de código:** os códigos são sequenciais e coincidem com a posição de cadastro, o que dispensa uma estrutura de busca, o índice é obtido diretamente do código.
- **Capacidade fixa (50 livros):** definida estaticamente no `.space`, com verificação explícita antes de cada cadastro.
- **Sem persistência:** os dados vivem apenas em memória durante a execução, o que é coerente com o escopo do trabalho.

3. Testes

Os testes foram executados no simulador MARS:

O cenário abaixo cadastra três livros, lista o acervo, empresta o livro 1 e 3, confirma a mudança de estado, devolve o livro 1 e exercita os dois casos de erro (devolver um livro disponível, devolver um livro inexistente, emprestar um livro emprestado e emprestar um código inexistente). A saída a seguir **não foi editada**.

Escolha entre:

1. Cadastrar livro;

2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 1

Digite o título do livro: teste

Código: 1 - Livro cadastrado com sucesso!

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 1

Digite o título do livro: teste

Código: 2 - Livro cadastrado com sucesso!

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 1

Digite o título do livro: teste

Código: 3 - Livro cadastrado com sucesso!

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 4

Código: 1 | Disponível

Código: 2 | Disponível

Código: 3 | Disponível

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 2
Digite o código do livro: 1
Livro emprestado com sucesso!
Escolha entre:
1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 2
Digite o código do livro: 3
Livro emprestado com sucesso!
Escolha entre:
1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 3
Digite o código do livro: 1
Livro devolvido com sucesso!
Escolha entre:
1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 4
Código: 1 | Disponível
Código: 2 | Disponível
Código: 3 | Emprestado
Escolha entre:
1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 2
Digite o código do livro: 5
Código inválido!
Escolha entre:
1. Cadastrar livro;

2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 2

Digite o código do livro: 3

Livro já está emprestado!

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 3

Digite o código do livro: 5

Código inválido!

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção: 3

Digite o código do livro: 2

Livro não está emprestado!

Escolha entre:

1. Cadastrar livro;
2. Emprestar livro;
3. Devolver livro;
4. Listar livros;
5. Sair;

Sua opção:

Resumo dos casos verificados:

Caso testado	Resultado esperado	Resultado obtido
Cadastro de 3 livros	Códigos 1, 2 e 3 gerados	OK
Listagem inicial	Todos disponíveis	OK

Caso testado	Resultado esperado	Resultado obtido
Empréstimo do livro 1	"Livro emprestado com sucesso!"	OK
Listagem após empréstimo	Livro 1 emprestado	OK
Devolução do livro 1	"Livro devolvido com sucesso!"	OK
Listagem após devolução	Todos disponíveis	OK
Devolver livro disponível	"Livro nao esta emprestado!"	OK
Emprestar código inexistente (9)	"Codigo invalido!"	OK

4. Conclusão

O trabalho cumpre todos os requisitos da especificação: cadastro, listagem, empréstimo e devolução de livros, com as duas regras de negócio (não emprestar livro já emprestado e não devolver livro não emprestado) devidamente garantidas.

A principal dificuldade esteve no gerenciamento manual da memória em Assembly: calcular endereços dos vetores exige atenção ao tamanho de cada elemento - inteiros de 4 bytes em `codigos` (deslocamento com `sll` por 2) versus bytes em `status` (soma direta do índice). Outro ponto sensível foi manter uma **convenção de status consistente** entre todas as rotinas; uma divergência nesse ponto (disponível/emprestado representados por valores diferentes em rotinas distintas) gerava falha silenciosa na devolução, corrigida durante os testes.

De modo geral, o exercício reforçou o entendimento sobre chamadas de sistema (syscalls) do MARS, controle de fluxo com desvios condicionais e a manipulação explícita de vetores em memória, conceitos centrais na programação em baixo nível.

Como executar

Pré-requisitos: Java instalado e o arquivo `Mars4_5.jar` na raiz do projeto.

```
java -jar Mars4_5.jar main.asm # abre a interface gráfica do MARS
```

Ou diretamente pelo console:

```
java -jar Mars4_5.jar nc main.asm # nc = no console GUI, roda no terminal
```